

Control Systems — Course Notes

Printed version • Version: 26 September 2026

Where to write your answers: if you have access to a device, use the **online course notes** instead — you can type your answers into the answer panels and they are saved as you go. Otherwise, write your answers in your **jotter**, with the task and question number beside each answer.



Scan for the online course notes

Section 1: Electronic Control Systems

Almost every modern device — from a microwave oven to a car engine — uses an **electronic control system**. These systems allow devices to respond to the world around them, make decisions, and control outputs automatically.

Any electronic control system can be broken down into three parts:



Devices that combine **electronic control** with **moving mechanisms** are called **mechatronic** systems. A washing machine, robot arm or delivery drone are all mechatronic — electronics tells the mechanisms what to do.

Inputs and outputs

In general, all systems need inputs from the outside world to sense what is happening and need outputs to influence it.



Input devices (sensors)

- **Push switch / toggle switch** — detects a press or on/off
- **Light-dependent resistor (LDR)** — detects light level
- **Thermistor** — detects temperature
- **Microphone** — detects sound
- **Pressure pad** — detects weight or pressure
- **Tilt switch** — detects orientation
- **Infrared sensor** — detects IR signals or motion



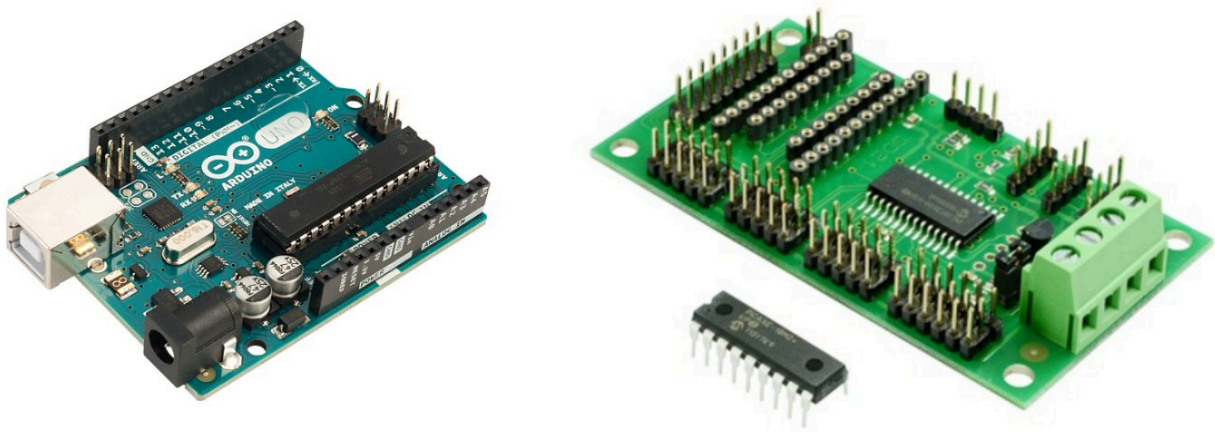
Output devices (actuators)

- **LED / lamp** — produces light
- **Buzzer / speaker** — produces sound
- **DC motor** — produces rotary motion
- **Solenoid** — produces linear motion
- **Relay** — switches higher-power circuits
- **7-segment display** — displays numbers
- **Servo motor** — produces controlled angular movement

Section 2: What is a Microcontroller?

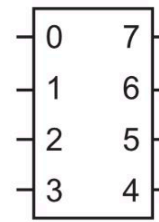
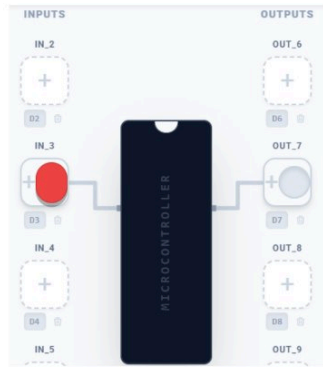
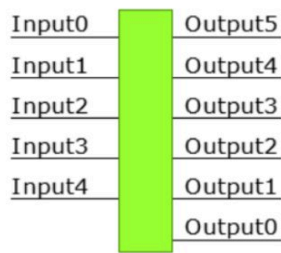
A **microcontroller** is often described as a *computer-on-a-chip*. It contains a processor, memory and input/output connections all in a single small, inexpensive package. Because of this, microcontrollers can easily be built into everyday products to make them more intelligent.

It is easy to think of a microcontroller only as a diagram or a symbol, but it is a real physical device — and different manufacturers package one very differently. The photograph below shows two boards commonly used in schools. On the left is an **Arduino Uno**: the main chip sits at the centre of the board, surrounded by the digital and analogue input/output pins along the edges, with a USB connector for programming and a power socket. On the right is a **PICAXE** project board, along with a bare PICAXE chip of the kind you'll find plugged into it — PICAXE chips are programmed in a similar way to an Arduino, but are typically mounted onto a simpler board that adds just the input/output connections and a power supply needed for a specific project.



Unlike a general-purpose computer, a microcontroller is typically **programmed to do one specific job**. The key advantage is flexibility — the same hardware can be reprogrammed to do something completely different simply by changing the software.

Away from the physical hardware, a microcontroller is drawn differently again depending on which software or exam paper you are looking at. The image below shows three examples: in **Yenka**, it appears as a simple green block with numbered inputs and outputs; in **NoSyntaxSim**, it appears as an on-screen component with input and output devices connected directly to it; and in a **QS examination or assignment**, it is drawn even more simply, as a plain rectangle with numbered pins down each side. All three are showing the same idea — inputs on one side, outputs on the other — just presented in the style of whichever tool you're using.



Hard-wired circuits

It is possible to produce electronic control circuits without microcontrollers, but they usually need a lot more parts and are significantly more complex. A control circuit without a programmable element is called a **hard-wired circuit**.

Advantages of microcontrollers

- **Reliability** — fewer separate components means fewer things to go wrong
- **Compact** — one chip replaces many separate components, saving space
- **Flexible** — reprogramming changes what the product does without changing hardware
- **Rapid development** — changes to the product are made in software, not by redesigning the circuit
- **Low cost** — mass production makes microcontrollers very inexpensive

A microcontroller is advantageous because the control sequence can be changed by reprogramming rather than redesigning the hardware. This reduces cost and development time.

Task 1 — Control systems

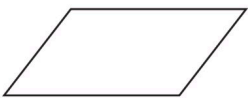
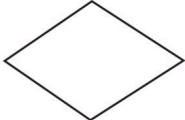
1. Explain why an electronic engineer might choose to use a microcontroller rather than a hard-wired circuit for a control system.
2. Name three devices that contain a microcontroller. For each, explain why the microcontroller is useful in that product.
3. Responsive interactive toys use sensors to react to their environment and users' inputs. They often mimic pet animals - making life like noises and movements. List three input sensors that a toy like this might have and describe what each one detects.

Section 3: Flowcharts

A **flowchart** is a graphical way of planning and communicating a control sequence. It shows exactly what the microcontroller should do, in what order, and under what conditions. Flowcharts are used in the QS assignment and exam — you must be able to read, draw, correct and simulate them.

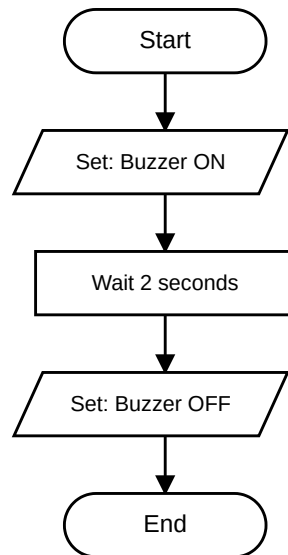
Standard QS flowchart symbols

A flowchart determines the order in which instructions are followed by the microcontroller — it is the plan the program must execute, step by step. Each type of instruction (starting/stopping, inputting/outputting, processing, or deciding) has its own standard box shape, so that anyone reading the flowchart can immediately recognise what kind of instruction each box represents, without needing to read every word.

Terminator symbol		Used for the start and end of a main program or sub-procedure.
Line symbol		Shows the direction of program flow. For flow down or to the right, an arrow is not needed. For flow upwards or to the left, arrows are added.
Input/Output		Used to control outputs or to show that data is being received.
Process symbol		Used for operations which take place within the microcontroller, for example a delay.
Decision symbol		Program flow is determined by a “yes” or “no” answer to the question in the box.
Sub procedure symbol		Used to call a sub-procedure.

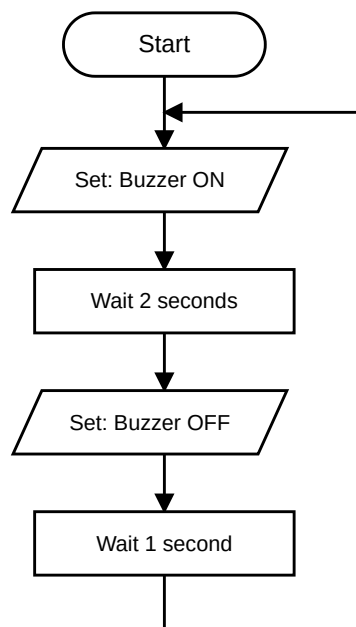
These symbols are taken from the QS data booklet.
(although the sub procedure symbol is not assessed at National 5 level)

A flowchart could simply be a single run of commands from start to finish, ending once the last instruction has been carried out. The example below starts the program, turns an output ON, waits for a set time, then turns the output OFF before stopping.



Note that a delay must always include its units. Wait 2 seconds, Wait 2s, Wait 2000 milliseconds or Wait 2000ms would be acceptable in the example above. Remember 1 second = 1000 milliseconds.

However, a flowchart like this would only run through once before stopping. In practice, it is far more common for microcontroller programs to run continuously — repeating the same sequence over and over. This is called a **continuous loop**. The example below shows this using the flow line symbol, which loops back to an earlier point in the sequence instead of reaching STOP, so the sequence repeats indefinitely.



Task 3 — Flowchart symbols and terminology

1. State the name and purpose of each flowchart symbol: (a) diamond, (b) parallelogram, (c) pill/rounded rectangle, (d) rectangle.
2. What is the correct QS term for a flowchart that repeats continuously? Why is "closed loop" incorrect?
3. A student writes "Wait 2000" in a Process box. What is wrong with this, and how should it be written correctly?

Section 4: Drawing Flowcharts from a Specification

In the QS exam and assignment you will be given a written specification and asked to draw a flowchart. Read the specification carefully — each step becomes one or more boxes in the flowchart.

What is a pin?

A microcontroller connects to the outside world through a number of **pins** along its edges — each one is a physical connection point that can be wired to an input device (like a switch) or an output device (like an LED). Every pin has its own number, such as **D8** or **D2**, so that a flowchart or program can refer to exactly the right connection. From here on, worked examples and questions will refer to specific pin numbers — always check the pin table given with a task to see which device is connected to which pin.

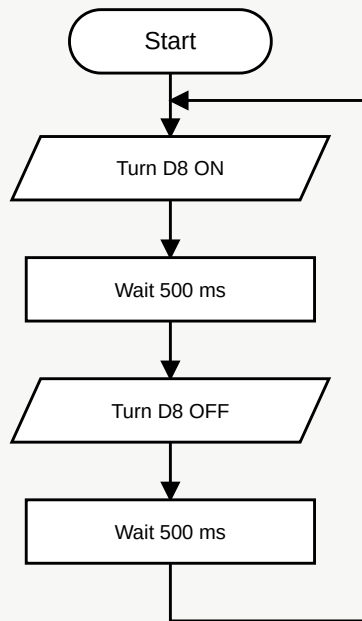
Method

1. Read the full specification first
2. Identify all inputs and outputs from the pin table
3. Draw a START terminal
4. Work through the specification step by step — one action per box
5. Use Decision boxes for any "wait until..." or "if... then..." conditions
6. Use Process boxes for all delays (with units)
7. Use In/Out parallelograms to turn outputs ON or OFF
8. Add a continuous loop arrow back to the start if the sequence repeats

WORKED EXAMPLE — FLASHING LED

SPECIFICATION

1. Turn LED (D8) ON
2. Wait 500 ms
3. Turn LED (D8) OFF
4. Wait 500 ms
5. Repeat continuously



Note the continuous loop arrow returning from the last box to the "Turn D8 ON" step.

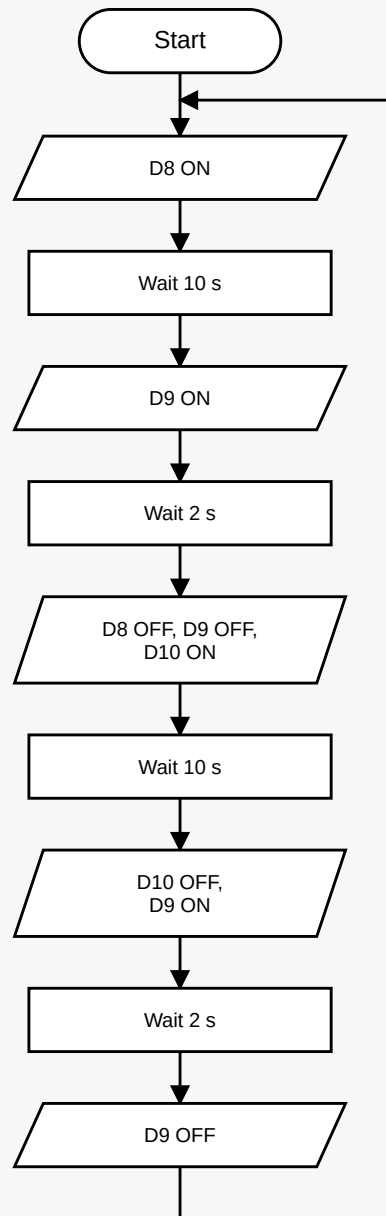
WORKED EXAMPLE — TRAFFIC LIGHTS

SPECIFICATION — PIN TABLE

Pin	Direction	Device
D8	OUTPUT	Red LED
D9	OUTPUT	Amber LED
D10	OUTPUT	Green LED

SEQUENCE

1. Red ON — Wait 10 s
2. Red ON, Amber ON — Wait 2 s
3. Red OFF, Amber OFF, Green ON — Wait 10 s
4. Green OFF, Amber ON — Wait 2 s
5. Amber OFF — repeat from step 1



NoSyntaxSim — Simulate the Worked Examples

This interactive simulator is in the online course notes. Scan the QR code, or use a device, to open it.

Task 4 — Drawing flowcharts from a specification

For each task: sketch your flowchart, then build and simulate it in NoSyntaxSim.

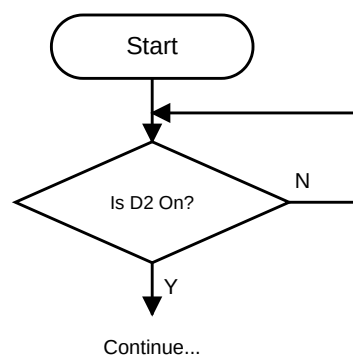
1. Design a system that makes a lamp (D8) flash on and off continuously. Each on and off period should last 0.5 s.
2. Build and simulate the traffic lights sequence shown in the worked example above. Pin table: D8 = Red, D9 = Amber, D10 = Green. The sequence must repeat as a continuous loop, not stop after one cycle.
3. A microcontroller is used to control a microwave oven using the sequence below. Complete a flowchart and then simulate the system. NoSyntaxSim has no magnetron or turntable component — represent the magnetron with an LED (D10) and the turntable motor with the standard motor component (D9).

MICROWAVE SPECIFICATION

1. Wait until start button (D2) AND door sensor (D3) are both ON
2. Turn light (D8) ON, turntable motor (D9) ON, magnetron (D10) ON
3. Wait 10 s
4. Turn magnetron (D10) OFF
5. Wait 1 s — Turn turntable (D9) OFF
6. Turn buzzer (D11) ON — Wait 0.5 s — Turn buzzer (D11) OFF
7. Turn light (D8) OFF — STOP

Section 5: Using Inputs — Decision Boxes

Most real control systems respond to sensor inputs. A **decision box** (diamond) checks whether an input is ON or OFF and branches the flowchart accordingly.



Waiting for an input

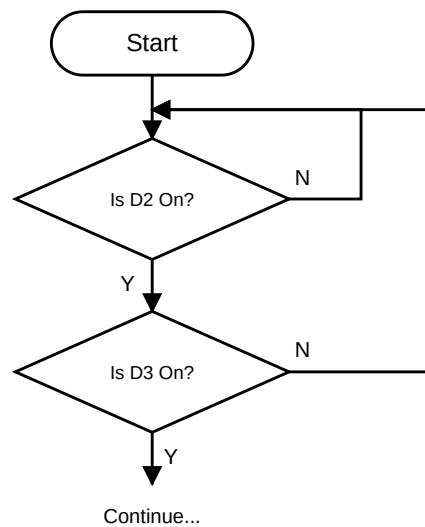
To make the program wait until a button is pressed, use a decision box that loops back to itself on the NO branch:

Is D2 ON? → **NO**: loop back to the same question → **YES**: continue to next step

This is called a **polling loop** — the microcontroller keeps checking the input until it becomes active.

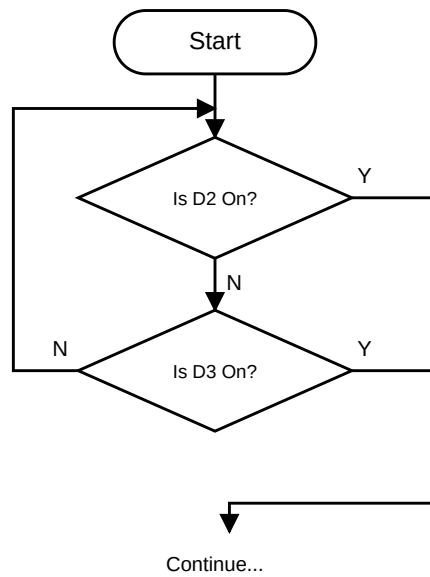
AND control — both inputs must be active

When **two conditions must both be true** before continuing, two decision boxes are placed in series. The YES output of the first leads into the second decision. Only when both return YES does the program continue — and if the second condition is not met, the flowchart loops all the way back to the **first** decision, not just the second. This matters: without looping back to the top, the two inputs could be triggered one after another (rather than together) and the program would still be allowed through.



OR control — either input can be active

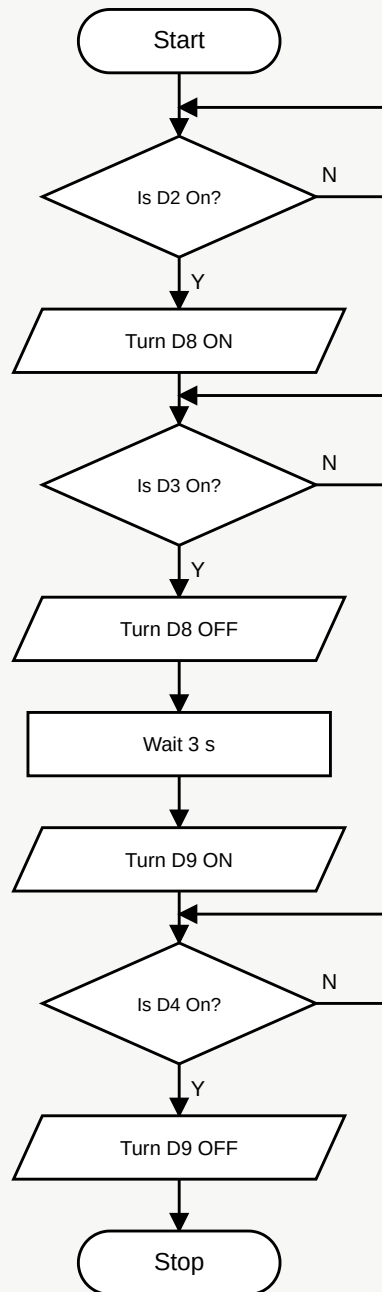
When **either of two conditions being true** is enough to continue, two decision boxes are placed in parallel. The YES output of either one leads to the same next step.



WORKED EXAMPLE — AUTOMATIC DOOR WITH INPUTS

SPECIFICATION

1. Wait until open button (D2) is pressed
2. Turn motor (D8) ON — door opens
3. Wait until door-open button (D3) is pressed — door fully open
4. Turn motor (D8) OFF
5. Wait 3 s
6. Close door — turn motor (D9) ON (reverse direction)
7. Wait until door-closed button (D4) is pressed
8. Turn motor (D9) OFF — STOP (or loop)



WORKED EXAMPLE — MOTOR DIRECTION CONTROL

Many control systems need to drive a motor in **two directions** — for example a buggy that drives forwards then reverses. In NoSyntaxSim, two separate output pins control direction: one pin drives the motor forwards, the other drives it backwards. Only one should ever be ON at the same time.

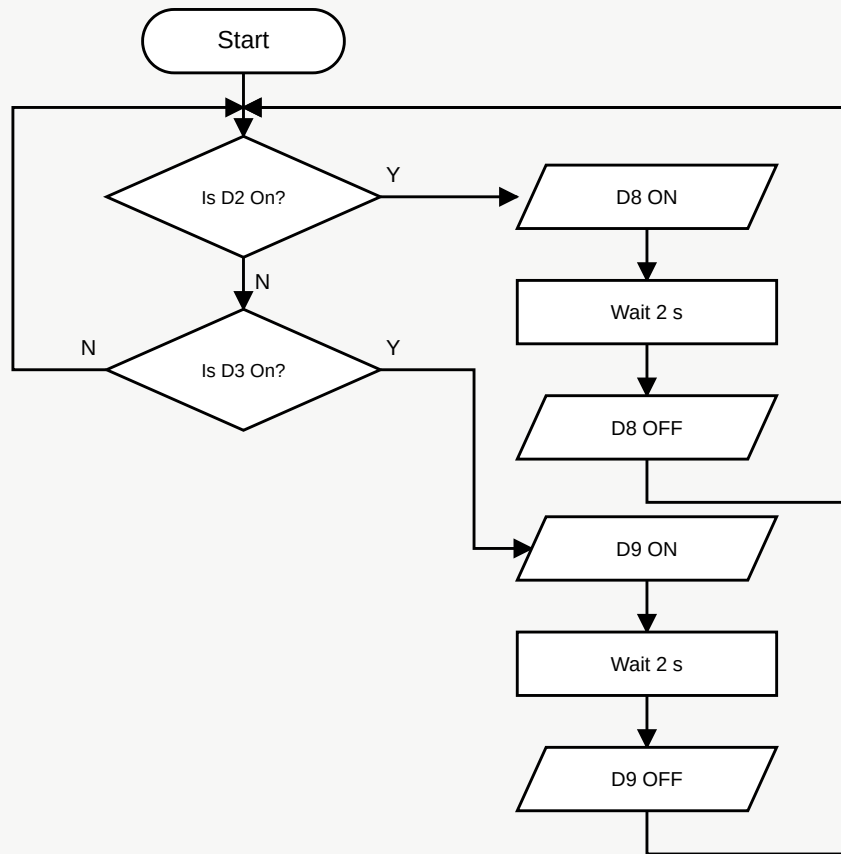
SPECIFICATION — PIN TABLE

Pin	Direction	Device
D2	INPUT	Forward button
D3	INPUT	Reverse button
D8	OUTPUT	Motor forwards
D9	OUTPUT	Motor backwards

SEQUENCE

1. Wait until forward button (D2) OR reverse button (D3) is pressed
2. If D2 is pressed → turn D8 ON (motor forwards) — Wait 2 s — turn D8 OFF
3. If D3 is pressed → turn D9 ON (motor backwards) — Wait 2 s — turn D9 OFF
4. Repeat (continuous loop)

 **Important:** Never turn D8 and D9 ON at the same time — this would cause a short circuit in a real motor driver. The flowchart must always turn one OFF before turning the other ON.



NoSyntaxSim — Inputs and Decision Boxes

This interactive simulator is in the online course notes. Scan the QR code, or use a device, to open it.

Task 5 — Inputs and decision boxes

1. A traffic light system stays on green until a pedestrian presses a button (D2). When pressed, the lights change: Amber (D9) for 2 s, then Red (D8) for 5 s, then Red+Amber together for 2 s, then back to Green (D10). Draw the flowchart and simulate it.
2. A security door in a bank must only open when two buttons (D2 AND D3) are both pressed together. It must close when a third button (D4) is pressed. Use the motor component (D8) to simulate the door opening and closing. Design and simulate the system.
3. A burglar alarm must sound (D8) when either a pressure pad (D2) is pressed OR a window sensor (D3) is activated. Design and simulate the system.
4. A heater (D8) must switch on when the temperature drops (D2 = ON when cold). It must be switched off by a separate reset button (D3). NoSyntaxSim has no heater component — use an LED (D8) to represent the heater. Design and simulate the system.

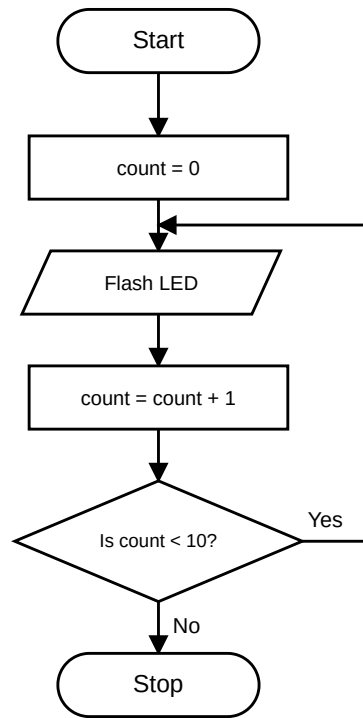
Section 6: Finite (Counted) Loops

A **finite loop** — also called a **fixed loop**, a **counted loop**, or a **for loop** — repeats a sequence a set, predetermined number of times decided in advance, then stops and moves on. This is different from a continuous loop, which has no predetermined end and repeats indefinitely.

In programming and in the QS exam, this construct is called a **for...next loop**, but you may also see it called a **fixed loop** — both describe the same thing: a loop that runs for a set number of repeats, decided in advance. The loop runs **FOR** a set number of times; **NEXT** sends the program back to repeat the loop again, until the count condition is met — only then does the program move on to the instructions after the loop. You must be able to recognise these terms and construct a flowchart that represents them.

To create a for...next loop in a flowchart, a **variable** (e.g. *count*) is used to keep track of how many times the loop has run:

1. **Initialise** the variable: $count = 0$
2. Perform the repeated action (e.g. flash an LED)
3. **Increment** the variable: $count = count + 1$
4. **Decide**: is $count < \text{target number}$? YES → repeat. NO → exit loop.



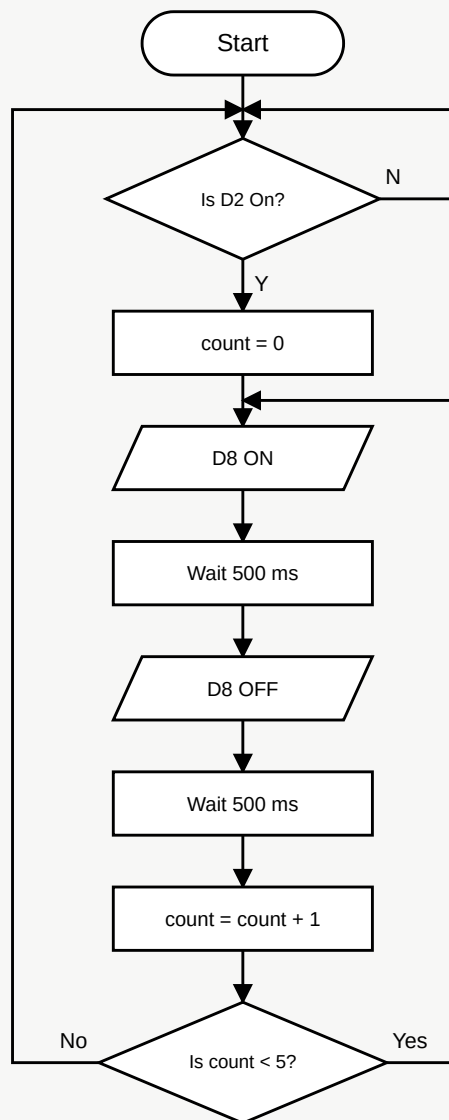
In NoSyntaxSim, drag the **For Loop** block from the left panel — it automatically creates the three required blocks (initialise, increment, decision) formatted to QS standards.

WORKED EXAMPLE — SCHOOL BELL SOUNDS 5 TIMES

SPECIFICATION

1. Wait until button (D2) is pressed
2. Sound buzzer (D8) 5 times — each buzz lasts 0.5 s with 0.5 s gap
3. Reset and wait for the button again

- 1 Set count = 0
- 2 Turn D8 ON → Wait 500 ms → Turn D8 OFF → Wait 500 ms
- 3 count = count + 1
- 4 Is count < 5? YES → back to step 2. NO → loop back to wait for button.





NoSyntaxSim — Finite Loops

This interactive simulator is in the online course notes. Scan the QR code, or use a device, to open it.

Task 6 — Finite loops

1. Design a system that makes a lamp (D8) flash exactly 10 times. Each flash must last 0.5 s. Draw the flowchart and simulate it.
2. A school bell (D8) must sound 6 times when a button (D2) is pressed. Once complete, the system should reset and be ready for the next button press.
3. A laser cutter must move between two sensors. A switch (D2) activates the system. The motor turns clockwise (D8 ON) until sensor 1 (D3) is triggered. It then turns anticlockwise (D9 ON) until sensor 2 (D4) is triggered. This cycle repeats 5 times before stopping.

Section 7: Reading and Correcting Flowcharts

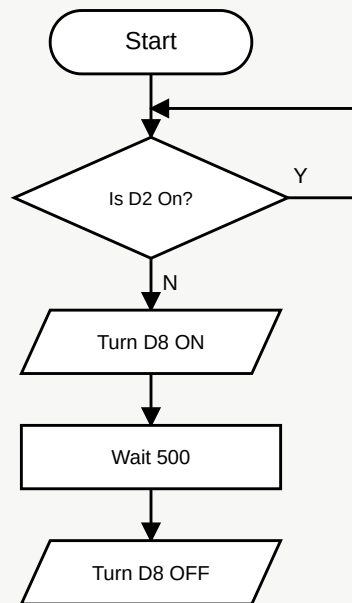
In the QS exam you will often be given a flowchart that contains **errors** and asked to identify and correct them.

Common errors include:

- Missing delay units (writing "500" instead of "500 ms")
- Wrong pin number used in an input or output box
- Decision box branching the wrong way (YES and NO swapped)
- Missing connection — the loop does not return to the correct point
- Wrong order of steps (e.g. turning a device ON after the delay instead of before)
- No START or STOP terminal
- Wrong flowchart symbol used (e.g. a rectangle instead of a diamond for a decision)

WORKED EXAMPLE — SPOT AND CORRECT THE ERRORS

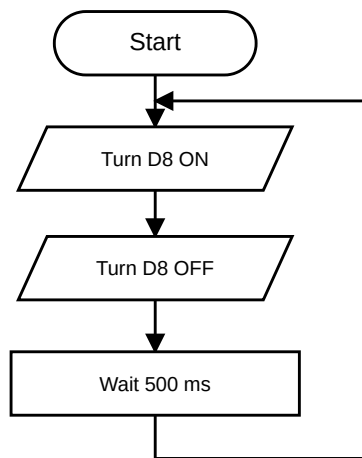
The flowchart below is supposed to flash a lamp (D8) when a moisture sensor (D2) is detected. Find two errors.



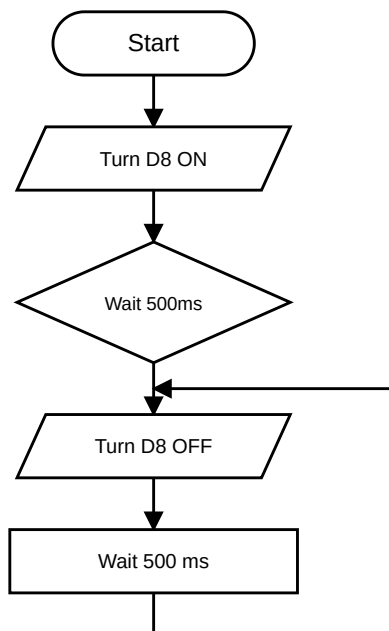
- 1 **Error 1:** The YES and NO branches on the decision box are swapped. The YES branch should lead to turning the LED ON, not loop back.
- 2 **Error 2:** The delay box reads "500" — this is missing the unit. It should read "Wait 500 ms".

Task 7 — Reading and correcting flowcharts

1. A flowchart is designed to flash an LED (D8) on and off, waiting 500 ms between each change. Study the flowchart below and identify the error. Explain what is wrong and how it should be corrected.



2. Another flowchart is also designed to flash an LED (D8) on and off continuously, waiting 500 ms between each change. Study the flowchart below — it contains two separate errors. Identify both, explain what is wrong with each, and describe the corrections needed.



Section 8: Inputs, Outputs and Pin Numbers

The microcontroller used in NoSyntaxSim is based on the **Arduino** platform. Inputs and outputs connect to numbered **digital pins** (D2–D13). In the QS exam, you will always be given a **pin table** telling you which pin connects to which input or output device.

If you are using a **PICAXE** chip instead, pins are numbered slightly differently — as **b.0** to **b.7** rather than D2–D13. QS flowchart questions and answers only ever use plain pin numbers (like D8), regardless of which hardware a class happens to be using, so that's the numbering to use when answering questions.

In any flowchart you draw, you **must** use the correct pin numbers from the pin table. The QS specifically requires pin numbers in flowcharts — do not just write "switch" or "LED".

Example pin table

Pin	Direction	Device
D2	INPUT	Push button (start switch)
D3	INPUT	Temperature sensor
D4	INPUT	Pressure pad
D5	INPUT	Reset button
D8	OUTPUT	Red LED
D9	OUTPUT	Green LED
D10	OUTPUT	Motor
D11	OUTPUT	Buzzer

Digital inputs

A **digital input** can only be logic 1 (ON) or logic 0 (OFF). A push button is digital — it is either pressed or not. In a decision box in a flowchart, a digital input is checked as: "Is D2 ON?"

Some inputs — like a thermistor or an LDR — can physically vary smoothly rather than switching cleanly between two states. At this level, you can still treat them as behaving digitally: they are simply LOW until a particular point is reached (e.g. a temperature or light level), then they become HIGH. In Yenka and in some tasks, this is exactly how these inputs are controlled and checked in a flowchart.

Task 8 — Inputs, outputs and pin tables

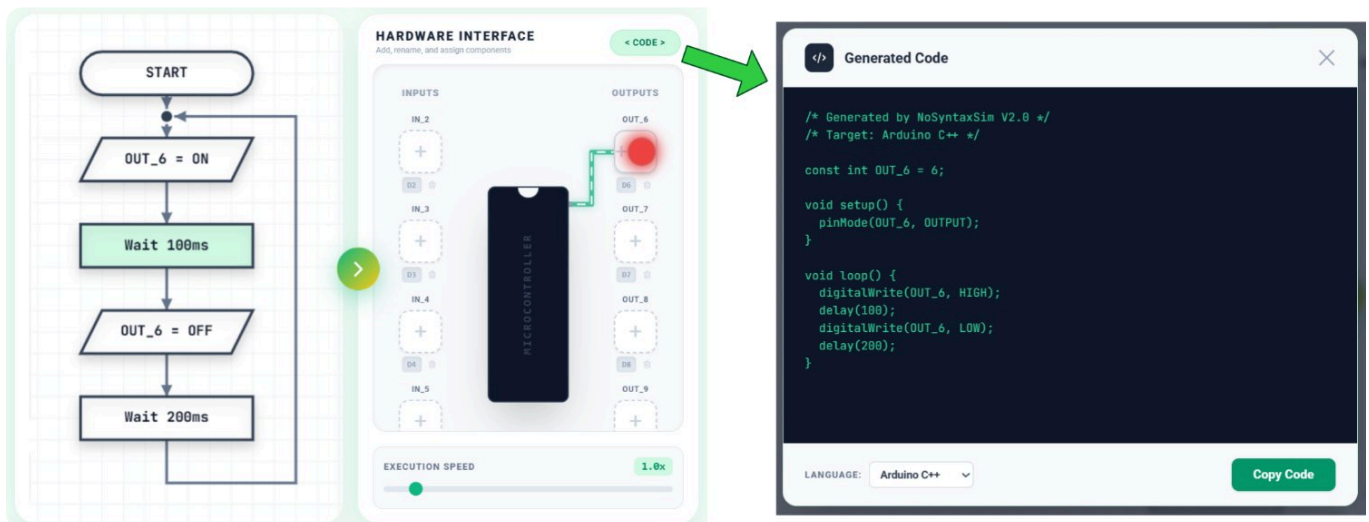
1. Using the example pin table above, identify which pin you would read in a flowchart to check if the pressure pad is activated.
2. Explain the difference between a digital input and an analogue input. Give one example of each.

Section 9: Generating Code

One of the most powerful features of NoSyntaxSim is that it can automatically convert your flowchart into real code — either **Arduino C++** or **pBASIC** (the language used by PICAXE chips), depending on which target you choose. This is the actual code that would run on a physical microcontroller.

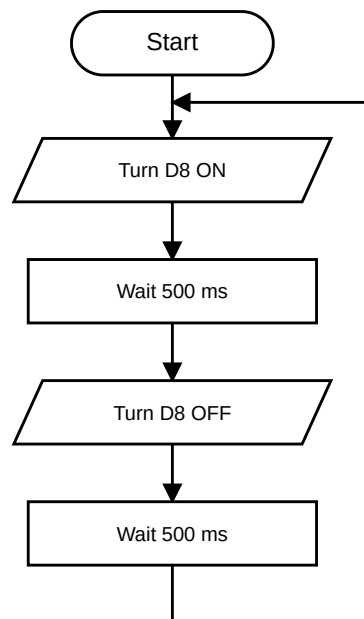
While you do not need to write or memorise code for the N5 exam, you may be asked to **read a short piece of code and correct an error in it**, so understanding the connection between a flowchart and the code it produces is useful preparation.

After building and testing your flowchart in NoSyntaxSim, click **GENERATE CODE** on the hardware panel. The code appears in a window, with a language selector for Arduino C++ or pBASIC — you can copy it and paste it directly into the Arduino IDE or PICAXE editor to run on real hardware.



Task 9 — From flowchart to code

1. Build the flashing LED flowchart shown below (the same one from the Flashing LED worked example) in NoSyntaxSim. Click GENERATE CODE. Look at the generated code and answer the following:



- What command or function does the generated code use to turn an output ON or OFF? (Note this is different in Arduino C++ and pBASIC.)
- What command or function is used to create a time delay?
- What part of the code makes the sequence repeat continuously? (In Arduino C++ this is the `loop()` function; in pBASIC this is the "main:" / "goto main" structure.)

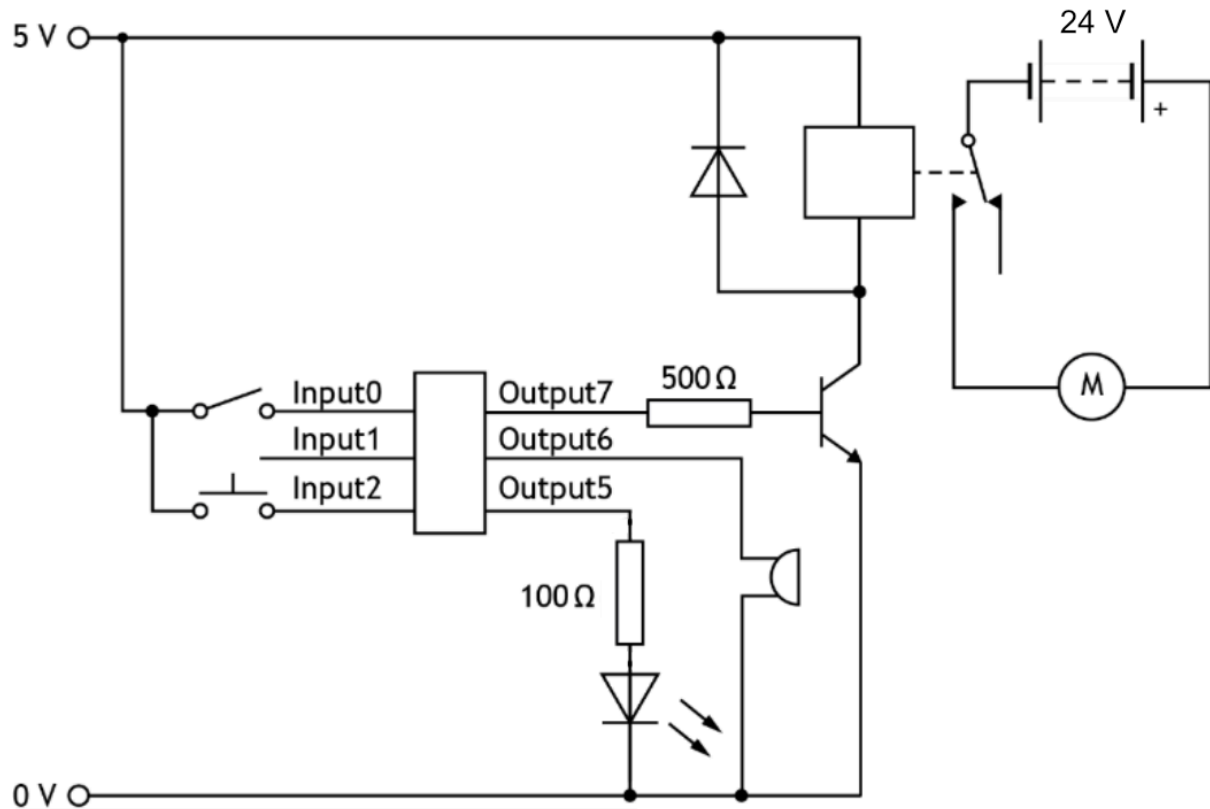
2. Explain in your own words why being able to generate code from a flowchart is useful for an engineer.

3. The code below is supposed to flash an LED on D8, waiting 500 ms between each change. Find and correct the error.

```
void loop() {  
  digitalWrite(8, HIGH);  
  digitalWrite(8, LOW);  
  delay(500);  
}
```

Output Drivers

A microcontroller pin can only supply a small current — not enough to directly drive a motor or solenoid. An **output driver** (typically a transistor or relay) amplifies the signal to power the output device. In exam circuits, a **transistor driver circuit** is commonly shown between the microcontroller output pin and the output device.



This circuit uses three outputs. **Output5** (LED) and **Output6** (buzzer) both draw very little current, so they run directly from the microcontroller's own supply. **Output7** drives a transistor, which switches a relay coil — the relay's separate contacts then control a motor circuit running from a much higher voltage (24 V), since that's more current and voltage than the microcontroller or transistor could handle directly. The diode across the coil protects the transistor from the voltage spike produced when the relay switches off.

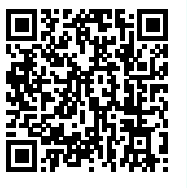
Output drivers — check your understanding

1. Explain why a transistor driver is needed between the microcontroller and a motor output.

Section 10: Design Challenges

For each challenge below, follow the full engineering process: read the specification carefully, design a flowchart, then build and simulate it in NoSyntaxSim. Test it against every point in the specification, and once it's working, export your solution and paste it into your notes as a record of your finished system.

How to approach these: Design your flowchart first — don't just start dragging blocks into NoSyntaxSim. Build it, then test every condition in the specification in turn (not just the "happy path"). If something doesn't work, work out which part of your flowchart is responsible before changing anything. Finally, evaluate: does your system fully meet the specification, and is there anything you'd improve given more time?



NoSyntaxSim — Design Challenge Workspace

This interactive simulator is in the online course notes. Scan the QR code, or use a device, to open it.

Challenge 1 — Fire alarm for a Cairngorms village hall

A village hall in the Cairngorms is having a control system fitted ahead of the winter ceilidh season, so the caretaker can arm the fire alarm overnight without needing a full commercial installation.

Pin	Direction	Device
D2	INPUT	Master arm switch
D3	INPUT	Heat sensor
D4	INPUT	Manual alarm button
D5	INPUT	Reset switch
D8	OUTPUT	Alarm

- i

The master switch (D2) must be pressed to arm the system.
- ii

Once armed, if the heat sensor (D3) is activated OR the manual alarm button (D4) is pressed, the alarm (D8) sounds.
- iii

The alarm continues to sound until the reset switch (D5) is pressed.
- iv

After reset, the system returns to waiting in the armed state.

Challenge 1 — Fire alarm

Design, build and simulate the fire alarm system described above. Test each part of the specification separately, then evaluate your finished system against it.

Challenge 2 — Starting signal for the Braemar hill races

The organisers of a Highland Games hill race want an automated starting signal, similar to the red-amber-green sequence used to start races at professional athletics meetings, so a single steward can start each wave of runners consistently.

Pin	Direction	Device
D2	INPUT	Start button
D8	OUTPUT	Red LED
D9	OUTPUT	Amber LED
D10	OUTPUT	Green LED
D11	OUTPUT	Buzzer

- i Wait until the start button (D2) is pressed.
- ii Red LED (D8) ON — wait 3 s.
- iii Red (D8) OFF, Amber (D9) ON — wait 1 s.
- iv Amber (D9) OFF, Green (D10) ON, Buzzer (D11) ON — wait 0.5 s.
- v Buzzer (D11) OFF — Green LED stays ON.
- vi The system waits for the next race (continuous loop back to step i).

Challenge 2 — Race start signal

Design, build and simulate the race starting signal described above. Test that the sequence times are correct and that it's ready to go again for the next race.

 **Higher-level content:** Challenges 3 and 4 below involve arithmetic on a variable (adding and subtracting values), which goes beyond the N5 course specification. They're included as stretch challenges — if you're working towards N5, you don't need to complete these to achieve your award, though most students are capable of having a go with some support.

Challenge 3 — Visitor counter for V&A Dundee

V&A Dundee wants a simple visitor counter fitted at a gallery entrance, so staff can see at a glance how many people are inside and be alerted if the room gets too busy.

This challenge combines decision boxes, a counted variable and two inputs.

Pin	Direction	Device
D2	INPUT	Entry sensor (ON when a person enters)
D3	INPUT	Exit sensor (ON when a person leaves)
D8	OUTPUT	Display / counter output
D9	OUTPUT	Alarm (sounds when the gallery is full)

- i Set count = 0.
- ii Check the entry sensor (D2) — if ON, add 1 to count.
- iii Check the exit sensor (D3) — if ON, subtract 1 from count.
- iv If count > 10 (gallery full), turn the alarm (D9) ON, otherwise turn it OFF.
- v Output the count value to the display (D8).
- vi Repeat continuously.

Challenge 3 — Visitor counter

Design, build and simulate the visitor counter described above. Test that the count rises and falls correctly, and that the alarm behaves correctly at the capacity limit.

Challenge 4 — Can dispenser for a school in Fort William

A school in Fort William is fitting a coin-operated can dispenser in the common room and needs a control system that accepts 10p and 20p coins, dispenses a can once enough money has been inserted, and gives change if too much is paid.

This challenge combines a for...next counted loop with multiple inputs and a sequenced output.

Pin	Direction	Device
D2	INPUT	10p coin sensor
D3	INPUT	20p coin sensor
D4	INPUT	Dispense button
D8	OUTPUT	Can dispenser motor
D9	OUTPUT	Change/reject indicator light

- i

Set total = 0.
- ii

Wait for a coin — if D2 is pressed (10p) add 10 to total; if D3 is pressed (20p) add 20 to total.
- iii

Repeat step ii until the dispense button (D4) is pressed.
- iv

If total $\geq$ 30p: turn D8 ON (dispense can) — wait 1 s — turn D8 OFF.
- v

If total $\neq$ 30p exactly: flash D9 (change light) 3 times — each flash 0.5 s ON, 0.5 s OFF.
- vi

Reset total to 0 and repeat.

 Challenge 4 — Can dispenser

Design, build and simulate the can dispenser described above. Test it with exact payment, overpayment and underpayment, then evaluate whether your system handles all three correctly.